

Spring Testing Exercises

Exercise 1: Basic Unit Test for a Service Method

Task: Write a unit test for a service method that adds two numbers.

Service:

```
@Service
public class CalculatorService {
    public int add(int a, int b) {
        return a + b;
    }
}
```

Test:

```
import static org.junit.jupiter.api.Assertions.assertEquals;
import org.junit.jupiter.api.Test;
public class CalculatorServiceTest {
    @Test
    void testAdd() {
        CalculatorService service = new CalculatorService();
        int result = service.add(10, 20);
        assertEquals(30, result);
    }
}
```

Exercise 2: Mocking a Repository in a Service Test

Task: Test a service that uses a repository to fetch data.

Entity:

```
@Entity
public class User {
    @Id
    private Long id;
    private String name;
    // getters and setters
}
```

Repository:

```
public interface UserRepository extends JpaRepository<User, Long> {
}
```

Service:

```
@Service
public class UserService { @Autowired
    private UserRepository userRepository;
    public User getUserById(Long id) {
        return userRepository.findById(id).orElse(null);
    }
}
```

Test:

```

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;
import java.util.Optional;
import org.junit.jupiter.api.Test;
public class UserServiceTest {
    @Test
    void testGetUserById() {
        UserRepository repository = mock(UserRepository.class);
        User user = new User();
        user.setId(1L);
        user.setName("John");
        when(repository.findById(1L)).thenReturn(Optional.of(user));
        UserService service = new UserService();
        service.userRepository = repository;
        User result = service.getUserById(1L);
        assertEquals("John", result.getName());
    }
}

```

Exercise 3: Testing a REST Controller with MockMvc

Task: Test a controller endpoint that returns a user.

Controller:

```

@RestController
@RequestMapping("/users")
public class UserController {
    @Autowired
    private UserService userService;
    @GetMapping("/{id}")
    public ResponseEntity<User> getUser(@PathVariable Long id) {
        return ResponseEntity.ok(userService.getUserById(id));
    }
}

```

Test:

```

import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.boot.test.mock.mockito.MockBean;
import org.springframework.test.web.servlet.MockMvc;

```

```

@WebMvcTest(UserController.class)
public class UserControllerTest {

```

```

@Autowired
MockMvc mockMvc;
@MockBean
UserService userService;
@Test
void testGetUser() throws Exception {
    User user = new User();
    user.setId(1L);
    user.setName("John");
    when(userService.getUserById(1L)).thenReturn(user);
    mockMvc.perform(get("/users/1"))
        .andExpect(status().isOk())
        .andExpect(jsonPath("$.name").value("John"));
}
}

```

Exercise 4: Integration Test with Spring Boot

Task: Write an integration test that tests the full flow from controller to database.

Test:

```

import static org.assertj.core.api.Assertions.assertThat;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
@SpringBootTest
public class UserIntegrationTest {
    @Autowired
    UserRepository repository;
    @Test
    void testDatabase() {
        User user = new User();
        user.setName("Alice");
        repository.save(user);
        assertThat(repository.findAll()).isNotEmpty();
    }
}

```

Exercise 5: Test Controller POST Endpoint

Task: Test a POST endpoint that creates a user. **Controller:**

```

@PostMapping
public ResponseEntity<User> createUser(@RequestBody User user) {
    return ResponseEntity.ok(userService.saveUser(user));
}

```

Test:

```

import static org.mockito.Mockito.when;

```

```

import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;
import com.fasterxml.jackson.databind.ObjectMapper;

@Test
void testCreateUser() throws Exception {
    User user = new User();
    user.setId(1L);
    user.setName("John");
    when(userService.saveUser(any(User.class))).thenReturn(user);
    mockMvc.perform(post("/users")
        .contentType("application/json")
        .content(new ObjectMapper().writeValueAsString(user)))
        .andExpect(status().isOk())
        .andExpect(jsonPath("$.name").value("John"));
}

```

Exercise 6: Test Service Exception Handling

Task: Test how a service handles a missing user.

Test:

```

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;
import java.util.Optional;

@Test
void testUserNotFound() {
    UserRepository repository = mock(UserRepository.class);
    when(repository.findById(1L)).thenReturn(Optional.empty());
    UserService service = new UserService();
    service.userRepository = repository;
    assertNull(service.getUserById(1L));
}

```

Exercise 7: Test Custom Repository Query

Task: Add and test a custom query method.

Repository:

```

public interface UserRepository extends JpaRepository<User, Long> {
    List<User> findByName(String name);
}

```

Test:

```

import static org.junit.jupiter.api.Assertions.*;
import java.util.List;

@Test
void testFindByName() {

    User user = new User();
}

```

```

    user.setName("John");
    repository.save(user);
    List<User> users = repository.findByName("John");
    assertEquals(1, users.size());
}

```

Exercise 8: Test Controller Exception Handling

Task: Add and test a `@ControllerAdvice` for handling exceptions.

Exception Handler:

```

@ControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(NoSuchElementException.class) public ResponseEntity<String>
    handleNotFound(NoSuchElementException ex) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body("User not found");
    }
}

```

Test:

```

@Test
void testUserNotFoundException() throws Exception {
    when(userService.getUserById(10L))
        .thenThrow(new NoSuchElementException());
    mockMvc.perform(get("/users/10"))
        .andExpect(status().isNotFound())
        .andExpect(content().string("User not found"));
}

```

Exercise 9: Parameterized Test with JUnit

Task: Use `@ParameterizedTest` to test multiple inputs.

Test:

```

import static org.junit.jupiter.api.Assertions.assertEquals;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;
public class CalculatorParameterizedTest {
    CalculatorService service = new CalculatorService();
    @ParameterizedTest
    @CsvSource({
        "2,3,5",
        "10,20,30",
        "5,5,10"
    })
    void testAdd(int a, int b, int expected) {
        assertEquals(expected, service.add(a, b));
    }
}

```